

Module 9 Java – Software Design Patter and Project

1. Software Design Patterns and Project (MVC + DAO)

Introduction to Software Design Patterns:

Que: 1

Definition and purpose of design patterns.

Ans:

Definition:

Design Pattern is a reusable solution to common problems that occur in software design.

Purpose:

The purpose of design patterns is to make code reusable, easy to understand, and easier to maintain while solving common programming problems efficiently.

Que: 2

Classification: Creational, Structural, and Behavioral patterns.

Ans:

Classification of Design Patterns:

1. Creational Patterns:

- These patterns deal with object creation and help create objects in a flexible way.
- **Example:** Singleton, Factory.

2. Structural Patterns:

- These patterns deal with how classes and objects are organized to form larger structures.
- **Example:** Adapter, Decorator.

3. Behavioral Patterns:

- These patterns focus on how objects communicate and interact with each other.
 - **Example:** Observer, Strategy.
-

Que: 3

Examples of popular patterns: Singleton, Factory, Observer, Decorator, etc.

Ans:

Examples of Popular Design Patterns:

1. Singleton:

- Ensures that a class has only one object and provides a global access point to it.

2. Factory:

- Provides a method to create objects without specifying the exact class.

3. Observer:

- Defines a one-to-many relationship where if one object changes, other dependent objects are notified automatically.

4. Decorator:

- Adds new functionality to an object dynamically without changing its structure.
-

Introduction to MVC Pattern

Que: 1

Model-View-Controller (MVC) architecture explained.

Ans:

MVC is a design pattern used to separate an application into three main components.

1. Model:

- Handles the data and business logic of the application.

2. View:

- Displays the data to the user (UI).

3. Controller:

- Receives user input and controls the interaction between Model and View.
-

Que: 2

Separation of concerns and how MVC helps in structuring applications.

Ans:

❖ **Separation of Concerns:**

Separation of concerns means dividing a program into different parts so that each part handles a specific task.

➤ **How MVC Helps:**

In MVC architecture, the application is divided into Model, View, and Controller, where:

- **Model** manages data and business logic.
 - **View** handles the user interface.
 - **Controller** manages user input and connects Model and View.
 - This separation makes the application easier to develop, maintain, and modify.
-

Introduction to Data Access Object (DAO)

Que: 1

Purpose of the DAO pattern in decoupling data access logic from business logic.

Ans:

❖ **Purpose of DAO Pattern:**

- The **DAO (Data Access Object) pattern** is used to separate data access logic from business logic in an application. It provides methods to perform database operations like insert, update, delete, and select.
 - This makes the code more organized, reusable, and easier to maintain because the database-related code is kept separate from the main application logic.
-

Que: 2

How DAO works in combination with MVC to interact with databases.

Ans:

❖ **DAO with MVC to Interact with Database**

- In MVC architecture, the DAO (Data Access Object) works with the Model to handle database operations.
 - **View:** Takes input from the user and displays data.
 - **Controller:** Receives the user request and processes it.
 - **Model:** Contains the application data.
 - **DAO:** Performs database operations like insert, update, delete, and select.

➤ **Working:**

User request → Controller → Model → DAO → Database → Response back to View.

- This structure keeps database code separate from business logic, making the application easier to manage and maintain.
-

2. Session Management (Session, Cookie, Hidden Form Field, URL Rewriting)

Session Management Overview

Que: 1

Why Session Management is Essential in Web Applications

Ans:

❖ **Why Session Management is Essential in Web Applications**

- Session management is essential because HTTP is a stateless protocol, meaning the server does not remember user information between requests.
 - Session management helps to maintain user data and track user activities during multiple requests, such as login information, shopping cart data, and user preferences. This allows web applications to provide a continuous and personalized user experience.
-

Que: 2

Difference between Client-side and Server-side Session Management

Ans:

❖ **Difference between Client-side and Server-side Session Management**

• **Client-side Session Management:**

Session data is stored on the client's browser, usually using cookies. The server reads this data when the user sends a request.

• **Server-side Session Management:**

Session data is stored on the server, and the client only keeps a session ID to identify the session.

• **Main Difference:**

Client-side stores data on the user's device, while server-side stores data on the server, making it more secure and reliable.

Session

Que: 1

Definition of a session and its importance in tracking user activity.

Ans:

❖ **Definition of Session:**

A session is a time period during which a web server keeps track of a user's activities while they interact with a web application.

➤ **Importance:**

A session is important because it helps the server identify and track the user across multiple requests, such as maintaining login information, user data, and shopping cart details.

Que: 2

How to Create, Retrieve, and Destroy Sessions in Java Servlets.

Ans:

❖ **How to Create, Retrieve, and Destroy Sessions in Java Servlets:**

1. Create Session:

- A session is created using the getSession() method of the request object.

`HttpSession session = request.getSession();`

2. Retrieve Session:

- To get an existing session, use:

`HttpSession session = request.getSession(false);`

3. Destroy Session:

- A session can be destroyed using the invalidate() method.

`session.invalidate();`

➤ These methods help manage user sessions in Java Servlet-based web applications.

Cookies

Que: 1

What cookies are and how they store small amounts of data on the client-side.

Ans:

• **Cookies:**

Cookies are small text files stored on the user's browser (client-side) by a web server.

• **Purpose:**

They store small amounts of data such as user preferences, login information, and session IDs. This data helps the website recognize the user and maintain session information when the user visits the website again.

Que: 2

Creating, reading, updating, and deleting cookies in Java servlets.

Ans:

❖ **Creating, Reading, Updating, and Deleting Cookies in Java Servlets**

1. Creating Cookie:

```
Cookie cookie = new Cookie("username", "John");
response.addCookie(cookie);
```

2. Reading Cookie:

```
Cookie cookies[] = request.getCookies();
for(Cookie c : cookies){
    if(c.getName().equals("username")){
        out.println(c.getValue());
    }
}
```

3. Updating Cookie:

```
Cookie cookie = new Cookie("username", "David");
response.addCookie(cookie);
```

4. Deleting Cookie:

```
Cookie cookie = new Cookie("username", "");
cookie.setMaxAge(0);
response.addCookie(cookie);
```

Hidden Form Fields

Que: 1

Explanation of hidden form fields and their role in passing data between pages.

Ans:

- **Hidden Form Fields:** Hidden form fields are input fields in an HTML form that are not visible to the user but are used to store and send data from one page to another.
 - **Role:** They help in passing information between web pages, such as user ID or session data, when the form is submitted to the server.
-

URL Rewriting

Que: 1

How URL rewriting can be used to track sessions when cookies are disabled.

Ans:

- **URL Rewriting:**

URL rewriting is a technique used to track user sessions by adding session information directly to the URL.

- **When Cookies Are Disabled:**

If cookies are disabled, the session ID is appended to the URL, so the server can identify the user in each request.

- **Example:**

<http://example.com/page?sessionId=12345>

➤ This helps the web application maintain session tracking without using cookies.

3. Project Covering Topics

Template Integration

Que: 1

What is template integration in web applications.

Ans:

- **Template Integration:**

Template integration is a technique used in web applications to combine a common layout (template) with dynamic content to generate web pages.

- **Purpose:**

It helps maintain a consistent design, reduces code duplication, and makes web pages easier to manage and update.

Que: 2

Importance of using pre-built templates for faster UI development

Ans:

➤ **Importance of Using Pre-built Templates for Faster UI Development**

- Pre-built templates help developers create user interfaces quickly without designing everything from scratch. They provide ready-made layouts and components, which saves time and effort.
 - Using templates also ensures consistent design, better user experience, and faster development of web applications.
-

Image Upload/Download

Que: 1

Steps to upload and download files in Java web applications.

Ans:

➤ **File Upload Steps:**

1. Create an **HTML form** with `enctype="multipart/form-data"`.
2. Use an **input type="file"** field in the form.
3. Create a **Servlet** to receive the uploaded file.
4. Use **request.getPart()** to get the file data.
5. Save the file on the **server folder**.

❖ **File Download Steps:**

1. Create a **Servlet** to handle the download request.

2. Set the **response content type**.
 3. Read the file from the **server location**.
 4. Write the file to **response output stream**.
 5. The browser will **download the file**.
-

Que: 2

Explanation of the multipart request and handling file uploads using MultipartConfig.

Ans:

❖ **Multipart Request and Handling File Uploads Using @MultipartConfig**

- **Multipart Request:**

A multipart request is a type of HTTP request used to send files and form data together from the client to the server. It is required when uploading files through an HTML form with enctype="multipart/form-data".

- **Handling File Uploads with @MultipartConfig:**

In Java Servlets, the @MultipartConfig annotation enables a servlet to receive and process file uploads.

➤ **Example:**

```
import jakarta.servlet.annotation.MultipartConfig;
import jakarta.servlet.http.*;
import java.io.*;
```

```
@MultipartConfig(fileSizeThreshold = 1024*1024, // 1MB
```

```
    maxFileSize = 1024*1024*5,    // 5MB
```

```
    maxRequestSize = 1024*1024*10) // 10MB
```

```
public class UploadServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
        throws IOException, ServletException {
```

```
        Part filePart = request.getPart("file"); // "file" is input name
```

```
        String fileName = filePart.getSubmittedFileName();
```

```
        String uploadPath = "C:/uploads/" + fileName;
```

```
        filePart.write(uploadPath); // Save file on server
```

```
        response.getWriter().println("File uploaded successfully: " + fileName);
```

```
    }
```

}

➤ **Key Points:**

- fileSizeThreshold → size after which file is written to disk.
 - maxFileSize → maximum allowed size for a single file.
 - maxRequestSize → maximum size for the entire request.
- This method simplifies file handling and ensures uploaded files are processed safely in web applications.
-

Mail Integration

Que: 1

How to send emails from a Java web application using JavaMail API.

Ans:

❖ **Steps:**

1. Add JavaMail Library:

- Include jakarta.mail.jar (or javax.mail.jar) in your project.

2. Set Mail Properties:

- Configure SMTP server details (host, port, authentication, TLS/SSL).

```
Properties props = new Properties();
```

```
props.put("mail.smtp.host", "smtp.gmail.com");
```

```
props.put("mail.smtp.port", "587");
```

```
props.put("mail.smtp.auth", "true");
```

```
props.put("mail.smtp.starttls.enable", "true");
```

3. Create a Session:

- Authenticate using your email and password (or app-specific password).

```
Session session = Session.getInstance(props, new javax.mail.Authenticator() {  
    protected PasswordAuthentication getPasswordAuthentication() {  
        return new PasswordAuthentication("your_email@gmail.com",  
        "your_password");  
    }  
});
```

4. Compose the Message:

```
Message message = new MimeMessage(session);
```

```
message.setFrom(new InternetAddress("your_email@gmail.com"));
```

```
message.setRecipients(Message.RecipientType.TO,  
InternetAddress.parse("recipient@example.com"));
```

```
message.setSubject("Test Email");
```

```
message.setText("This is a test email from Java web application.");
```

5. Send the Email:

```
Transport.send(message);
```

```
System.out.println("Email sent successfully!");
```

➤ Key Points:

- Use SMTP server of your email provider.
 - For Gmail, you may need an App Password.
 - JavaMail API supports text and HTML emails, as well as attachments.
- This allows your web application to send notifications, verification emails, or alerts directly from the server.
-

Que:2

Explanation of SMTP and how it's used for sending emails.

Ans:

- **Definition:**

SMTP (Simple Mail Transfer Protocol) is a protocol used to send emails from a client to a mail server or between mail servers over the internet. It defines the rules for how email messages are transmitted and routed.

- **How SMTP is Used for Sending Emails:**

1. The email client or application connects to the SMTP server (e.g., Gmail's smtp.gmail.com).
2. The client authenticates using a username and password.
3. The email message is transmitted to the SMTP server, which then forwards it to the recipient's mail server.
4. The recipient's mail server stores the message until the user retrieves it using IMAP or POP3.

- In Java web applications, SMTP is used with the JavaMail API to send emails programmatically.
-

OTP via Mail Integration

Que: 1

Introduction to OTP (One-Time Password) and its importance in enhancing security.

Ans:

OTP (One-Time Password)

- **Introduction:**

An OTP is a unique, temporary password that is valid for only one login session or transaction. It is usually sent to a user via SMS, email, or authenticator apps.

- **Importance in Security:**

- Prevents unauthorized access: Even if someone knows your regular password, they cannot log in without the OTP.
- Enhances authentication: Adds an extra layer of security (Two-Factor Authentication).
- Reduces fraud: Especially important for online banking, e-commerce, and sensitive transactions.

- OTP ensures that the user performing the action is genuine and authorized, making applications much more secure.
-

Que: 2

How to generate and send OTP via email for verification purposes.

Ans:

- ❖ **Steps:**

- 1. Generate OTP:**

- Use Java's Random class to create a numeric OTP.

- **Example:**

```
import java.util.Random;

int otpLength = 6;

Random random = new Random();

String otp = "";

for(int i = 0; i < otpLength; i++) {
    otp += random.nextInt(10); // generates digits 0-9
}

System.out.println("Generated OTP: " + otp);
```

- 2. Send OTP via Email (Using JavaMail API):**

○ **Example:**

```
import jakarta.mail.*;
import jakarta.mail.internet.*;
import java.util.Properties;
```

```
Properties props = new Properties();
props.put("mail.smtp.host", "smtp.gmail.com");
props.put("mail.smtp.port", "587");
props.put("mail.smtp.auth", "true");
props.put("mail.smtp.starttls.enable", "true");
```

```
Session session = Session.getInstance(props, new Authenticator() {
    protected PasswordAuthentication getPasswordAuthentication() {
        return new PasswordAuthentication("your_email@gmail.com", "your_password");
    }
});
```

```
try {
    Message message = new MimeMessage(session);
    message.setFrom(new InternetAddress("your_email@gmail.com"));
    message.setRecipients(Message.RecipientType.TO,
InternetAddress.parse("recipient@example.com"));
    message.setSubject("Your OTP Code");
    message.setText("Your OTP for verification is: " + otp);

    Transport.send(message);
    System.out.println("OTP sent successfully!");
} catch (MessagingException e) {
    e.printStackTrace();
}
```

3. Verification:

- Store the generated OTP temporarily on the server (e.g., session).
- Ask the user to enter the OTP received.

- Compare the entered OTP with the stored OTP to verify.
 - **Key Points:**
 - OTP should expire after a short time (e.g., 5 minutes).
 - Use secure channels like email or SMS.
 - Always handle exceptions to ensure emails are sent reliably.
 - This approach allows secure verification of users in web applications.
-

Online Payment Integration

Que: 1

Introduction to online payment gateways (e.g., PayPal, Stripe).

Ans:

- **Definition:**

An online payment gateway is a service that allows businesses to accept payments from customers over the internet securely. It acts as a bridge between the customer, the merchant, and the bank.

- **Examples:**

- PayPal – Widely used for online payments and international transactions.
- Stripe – Popular for integrating payment processing in web and mobile apps.

- **Purpose:**

- Process payments securely using credit/debit cards, UPI, or digital wallets.
- Authenticate transactions to prevent fraud.
- Provide seamless user experience for online shopping or subscription services.

- Payment gateways are essential for e-commerce, subscription services, and any online transaction-based applications.
-

Que: 2

How to integrate payment gateways into web applications

Ans:

❖ **Steps to Integrate a Payment Gateway (e.g., PayPal or Stripe):**

1. Create a Merchant Account:

- Register with the payment gateway provider (e.g., PayPal, Stripe).
- Get the API keys (public and secret keys) for your application.

2. Add Payment Gateway Library/SDK:

- Include the official SDK or API library in your project.
- For Java, you can use Maven or Gradle to include the SDK.

3. Create Payment Request:

- Collect payment details (amount, currency, customer info).
- Use the SDK to create a payment object or transaction.

4. Redirect User for Payment (if needed):

- Some gateways (like PayPal) redirect the user to their secure payment page.
- Others (like Stripe) allow direct payment integration on your site.

5. Handle Payment Response:

- The gateway sends a success or failure response after payment.
- Update your database and notify the user accordingly.

6. Secure Transactions:

- Use HTTPS for all payment-related communication.
- Validate responses from the gateway to prevent fraud.

➤ Example Flow:

User → Checkout Page → Payment Gateway → Payment Confirmation → Your Application

- This integration allows your web application to accept online payments safely and efficiently.
-

AJAX

Que: 1

Introduction to AJAX and its role in improving the user experience by enabling asynchronous requests.

Ans:

❖ Introduction to AJAX

➤ Definition:

AJAX (Asynchronous JavaScript and XML) is a technique that allows a web page to send and receive data from the server asynchronously, without reloading the entire page.

➤ Role in Improving User Experience:

- Enables faster interactions because only the required data is updated.
- Reduces page reloads, making the application feel smoother and more responsive.
- Commonly used in live search, form validation, chat apps, and dynamic content updates.

➤ **Example:**

Typing in a search box can show results instantly without reloading the page, thanks to AJAX.

- It makes web applications more interactive, efficient, and user-friendly.
-

Que: 2

Explanation of how AJAX works in combination with JavaScript and the server.

Ans:

How AJAX Works with JavaScript and the Server

1. User Action:

- The user triggers an event on the web page (e.g., clicking a button or typing in a search box).

2. AJAX Request (JavaScript):

- JavaScript creates an XMLHttpRequest (or uses fetch()) to send a request asynchronously to the server.

3. Server Processing:

- The server (e.g., a Java Servlet) receives the request, processes it (e.g., fetch data from a database), and sends a response back, usually in JSON, XML, or plain text.

4. AJAX Response Handling:

- JavaScript receives the server response and updates the web page dynamically without reloading the entire page.

➤ **Flow:**

User Action → JavaScript AJAX Request → Server Processing → Response → JavaScript Updates Page

- This asynchronous interaction makes the web application faster, smoother, and more responsive for users.
-